Unified Gradient-Boosted Intrusion Detection on Large-Scale CIC Traffic: Binary Detection and Multi-Class Attack Family Classification

Author Name

***Abstract*—Network intrusion detection systems must simultaneously maximize attack catch rate and provide actionable attack-type attribution. This paper reports a large-scale machine learning study based on a unified gradient boosting pipeline trained on a harmonized CIC traffic corpus (2017–2019), containing $9\,167\,581$ flow records and 59 tabular features. Two tasks are solved using identical train/validation/test partitions: (i) binary anomaly detection (Benign vs. Attack), and (ii) 8-class attack family classification (Benign, DDoS, DoS, Botnet, Bruteforce, Infiltration, Portscan, Webattack). LightGBM and XGBoost are evaluated under class-imbalance-aware training. For binary detection, both models achieve very high discrimination: LightGBM test AUC $0.996\,56$, F1 $0.972\,28$, Recall $0.958\,32$, Precision $0.986\,65$; XGBoost test AUC $0.996\,75$, F1 $0.971\,21$, Recall $0.958\,65$, Precision $0.984\,10$. For multi-class classification, LightGBM attains Macro F1 $0.779\,97$ and Weighted F1 $0.873\,39$, while XGBoost achieves Macro F1 $0.745\,24$ and Weighted F1 $0.872\,23$. Results show strong binary detection performance and competitive multi-class attribution, with persistent degradation on rare attack families due to extreme class imbalance. We discuss implications for operational deployment, threshold tuning, and hierarchy-aware post-processing.**

***Index Terms*—Intrusion Detection, Network Security, Gradient Boosting, LightGBM, XGBoost, Class Imbalance, Multi-Class Classification, CIC-IDS.**

I. Introduction

Modern enterprise and cloud networks require detection pipelines that are both sensitive (few missed attacks) and informative (attack-type context for triage). Classical signature-based IDS tools remain useful but can fail under novel or evolving attacks. As a result, supervised machine learning over network flow features has become a practical complement in many SOC and SIEM environments.

Two operational objectives frequently conflict. First, binary attack detection demands high recall and robust ranking under heavily imbalanced traffic. Second, multi-class attack attribution must preserve performance on rare classes, where minority groups can be statistically underrepresented by orders of magnitude. In this work, we evaluate whether a single, consistent gradient boosting pipeline can support both objectives at scale.

This study is based on a reproducible notebook implementation over a combined CIC traffic collection spanning 2017–2019. The resulting dataset is large enough to stress memory, class balance, and training-time concerns typical of real deployments.

This manuscript is a reproducibility-oriented report generated from an executable notebook workflow. Replace author and affiliation details before submission.

A. Contributions

The main contributions of this report are:

- A unified dual-task formulation on 9.17 million flows with shared stratified partitions for fair binary/multi-class comparison.
- A documented preprocessing workflow including schema normalization, data quality checks, and memory-aware downcasting.
- Side-by-side LightGBM/XGBoost evaluation with explicit imbalance handling.
- Quantitative analysis of global metrics and per-class behavior, including failure modes on minority classes.

II. Related Work

Tree-ensemble methods remain strong baselines for structured security telemetry. XGBoost introduced scalable second-order gradient boosting with regularized objective design and has been widely adopted in IDS settings [1]. LightGBM improved throughput and memory behavior through histogram-based learning and leaf-wise tree growth [2], often producing superior quality-time trade-offs on high-volume tabular datasets.

In intrusion detection research, the CIC benchmark family has become common for ML experimentation, including binary and multi-class tasks [3]. Recent studies increasingly emphasize not only aggregate accuracy but also class-aware metrics and operationally aligned evaluation under heavy imbalance [4]. Hyperparameter optimization frameworks such as Optuna provide efficient black-box tuning for boosting methods in this context [5].

III. Dataset and Problem Definition

A. Data Source

The notebook uses a harmonized CIC collection (2017/DoS2017/2018/2019) distributed through Kaggle and sourced from the Canadian Institute for Cybersecurity [6], [7]. Each row is a labeled bidirectional flow summary.

B. Dataset Scale and Split

Table I summarizes key scale values extracted from the executed notebook outputs.

TABLE I
DATASET AND SPLIT SUMMARY

Total rows	9 167 581
Total columns (post-cleaning)	59
Train rows	6 417 306
Validation rows	1 375 137
Test rows	1 375 138

TABLE II
MULTI-CLASS DISTRIBUTION (% OF FULL DATASET)

Class	Proportion (%)
Benign	78.39
DDoS	13.47
DoS	4.33
Botnet	1.59
Bruteforce	1.13
Infiltration	1.03
Webattack	0.03
Portscan	0.02

C. Class Composition

The corpus is strongly imbalanced. Multi-class proportions are shown in Table II. Binary relabeling yields 78.39% benign and 21.61% attack traffic.

D. Task Formulation

Two supervised tasks are solved on shared feature matrices:

- **Binary detection**: $y \in \{0, 1\}$ where 0 = Benign and 1 = Attack.
- **Multi-class attribution**: $y \in \{0, \ldots, 7\}$ using label encoding for 8 families.

The notebook label map is:

$$\{\text{Benign} : 0,\ \text{Botnet} : 1,\ \text{Bruteforce} : 2,\ \text{DDoS} : 3,\ \text{DoS} : 4,\ \text{Infiltration} : 5,\ \text{Portscan} : 6,\ \text{Webattack} : 7\}.$$

IV. METHODOLOGY

A. Preprocessing Pipeline

The preprocessing sequence is intentionally simple and production-oriented:

1) Normalize column names to lowercase snake-style tokens.
2) Replace infinite values with NaN and drop NaN rows when present.
3) Remove constant features (none removed in this run).
4) Downcast numeric types (e.g., float64 to float32) to reduce memory pressure.

Notebook output confirms zero missing and zero infinite values after checks. The implementation notes indicate an approximate memory reduction from ˜3.6 GB to ˜2.8 GB through downcasting.

B. Imbalance Handling

For binary LightGBM/XGBoost, class weighting uses:

$$\text{scale_pos_weight} = \frac{N_{\text{negative}}}{N_{\text{positive}}} = 3.63. \tag{1}$$

For multi-class training, balanced class weights are computed and applied as per-sample weights:

$$w_c = \frac{N}{K \cdot n_c}, \tag{2}$$

where N is total samples, K is number of classes, and n_c is class-c frequency.

C. Models and Training Setup

LightGBM (binary): objective `binary`, metric AUC, learning rate 0.05, `num_leaves`=31, `min_child_samples`=20, subsampling 0.8/0.8, 2000 rounds, early stopping 50. Output indicates no early stop reached, best validation AUC at iteration 2000.

XGBoost (binary): objective `binary:logistic`, metric AUC, learning rate 0.05, `max_depth`=8, `min_child_weight`=20, subsampling 0.8/0.8, 2000 rounds with early stopping callback. Validation AUC converges to ˜0.99679 by late rounds.

LightGBM (multi-class): objective `multiclass`, `num_class`=8, metric `multi_logloss`, learning rate 0.05, `num_leaves`=128, weighted samples, 2000 max rounds, early stopping 50. Best iteration reported at 76 with validation logloss 0.122398.

XGBoost (multi-class): objective `multi:softprob`, metric `mlogloss`, learning rate 0.05, `max_depth`=8, weighted DMatrix, 2000 max rounds, early stopping 50. Logged validation logloss approaches ˜0.114 in the 250–283 range.

Table III summarizes the explicit training settings captured in the notebook.

D. Metrics

Binary metrics:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \tag{3}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

AUC-ROC is used for threshold-independent ranking quality.

Multi-class metrics include Macro F1 and Weighted F1:

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{c=1}^{K} F_{1,c}, \quad F_1^{\text{weighted}} = \sum_{c=1}^{K} \frac{n_c}{N} F_{1,c}. \tag{5}$$

E. Implementation and Hardware Notes

The notebook probes GPU availability via `nvidia-smi`, reporting GPU presence in the execution environment. In the captured run, XGBoost uses CUDA histogram training for both binary and multi-class tasks, while the binary LightGBM run is explicitly configured to CPU in the final training cell. This mixed-device behavior is useful to report because it influences both wall-clock training time and deployment portability.

TABLE III
MODEL CONFIGURATION USED IN THE NOTEBOOK

Task	LightGBM	XGBoost
Binary objective	`binary`	`binary:logistic`
Primary metric	AUC	AUC
Learning rate	0.05	0.05
Tree complexity	`num_leaves`=31, `max_depth`=-1	`max_depth`=8, `min_child_weight`=20
Sampling	`subsample`=0.8, `colsample_bytree`=0.8	`subsample`=0.8, `colsample_bytree`=0.8
Imbalance handling	`scale_pos_weight`=3.63	`scale_pos_weight`=3.63
Boost rounds	2000 (best at 2000)	2000 (AUC stabilizes near late rounds)
Early stopping	50 rounds	50 rounds
Multi-class objective	`multiclass`, `num_class`=8	`multi:softprob`, `num_class`=8
Primary metric	`multi_logloss`	`mlogloss`
Learning rate	0.05	0.05
Tree complexity	`num_leaves`=128, `max_depth`=-1	`max_depth`=8, `min_child_weight`=20
Imbalance handling	class-weighted samples	weighted DMatrix
Boost rounds	early stop at iter 76 (best val logloss 0.122398)	monitored to 283+ rounds (val logloss ~0.114)

TABLE IV
VALIDATION CONVERGENCE SNAPSHOTS FROM LOGS

Model	Logged point	Value
LightGBM binary	iter 2000, val AUC	0.996586
XGBoost binary	iter 1999, val AUC	0.99679
LightGBM multi-class	best iter 76, val logloss	0.122398
XGBoost multi-class	iter 250, val logloss	0.11382
XGBoost multi-class	iter 283, val logloss	0.11415

TABLE V
BINARY TEST PERFORMANCE (ATTACK VS. BENIGN)

Metric	LightGBM	XGBoost
AUC	0.99656	0.99675
F1-score	0.97228	0.97121
Recall	0.95832	0.95865
Precision	0.98665	0.98410

F. Complexity Perspective

For histogram gradient boosting, one training iteration scales approximately with

$$\mathcal{O}(N \cdot F \cdot B), \tag{6}$$

where N is rows, F features, and B histogram bins (implementation dependent). With $N \approx 9.17 \times 10^6$ and dozens of features, memory traffic and split-search efficiency dominate practical runtime. This motivates the notebook design choice to downcast numeric dtypes and to keep feature engineering mostly flow-native rather than expanding dimensionality aggressively.

V. CONVERGENCE AND TRAINING DYNAMICS

The recorded logs show smooth and monotonic quality improvement in both binary learners. For LightGBM binary, validation AUC rises from roughly 0.99 at early rounds to 0.996586 at round 2000, without triggering early stopping. For XGBoost binary, validation AUC similarly climbs to 0.99679 near rounds 1900–1999.

For multi-class training, LightGBM reaches its best validation logloss earlier (iteration 76), while XGBoost continues gradual improvement over a larger horizon. This pattern is consistent with boosted-tree behavior under weighted minority classes: after early gains on dominant decision boundaries, late iterations allocate capacity to harder, sparse regions.

VI. EXPERIMENTAL RESULTS

A. Binary Detection

Table V reports held-out test metrics from the notebook comparison output.

Validation snapshots were similarly high:

- LightGBM: Validation AUC 0.99659, Recall 0.95813.
- XGBoost: Validation AUC 0.99679, Recall 0.95847, Precision 0.98488, F1 0.97149.

Observation. Both models are effectively tied in operational recall. XGBoost yields marginally higher AUC, while LightGBM slightly improves precision and F1 on the final test split.

B. Approximate Confusion-Matrix View at Fixed Threshold

Because the notebook reports precision/recall and test support, we can estimate confusion-matrix magnitudes for practical interpretation. The multi-class supports imply binary test totals of 1,077,929 benign and 297,209 attack samples.

Using reported test recall and precision:

- **LightGBM** approximately yields $TP \approx 284.8$k and $FN \approx 12.4$k, with $FP \approx 3.9$k.
- **XGBoost** approximately yields $TP \approx 284.9$k and $FN \approx 12.3$k, with $FP \approx 4.6$k.

These approximations reinforce a deployment-relevant point: both models miss a similar number of attacks at the current threshold, but LightGBM is somewhat more conservative in false alarms.

C. Multi-Class Attribution

Table VI summarizes aggregate multi-class performance.

Weighted F1 is close between models; however, LightGBM shows a clear Macro F1 advantage, indicating better behavior on less frequent classes.

TABLE VI
MULTI-CLASS OVERALL PERFORMANCE

Metric	LightGBM	XGBoost
Macro F1	0.77997	0.74524
Weighted F1	0.87339	0.87223

TABLE VII
PER-CLASS COMPARISON (TEST SPLIT)

Class	LightGBM		XGBoost		Support
	Recall	F1	Recall	F1	
Benign	0.74	0.85	0.74	0.85	1,077,929
Botnet	1.00	0.99	1.00	0.99	21,882
Bruteforce	1.00	1.00	1.00	1.00	15,347
DDoS	1.00	1.00	1.00	1.00	185,758
DoS	1.00	1.00	1.00	0.99	59,269
Infiltration	0.95	0.09	0.95	0.09	14,152
Portscan	0.95	0.80	0.98	0.69	353
Webattack	0.98	0.51	0.98	0.35	448

D. Per-Class Behavior

Table VII highlights class-level F1 and recall (test split support in the last column).

Interpretation. High recall but very low precision for Infiltration indicates substantial false positive contamination for that class. Similar behavior is visible for Webattack, where recall remains high but precision is poor, especially in XGBoost. These effects are typical under severe minority imbalance and overlapping feature manifolds.

E. Minority-Class Error Profile

The two classes with smallest support (Portscan and Webattack) illustrate opposite priorities:

- Recall remains very high (0.95–0.98), indicating low miss rate.
- Precision degrades substantially (e.g., Webattack precision 0.35 in LightGBM and 0.22 in XGBoost), indicating frequent over-assignment to rare labels.

For a SOC context, this means class predictions for these tails should be consumed as *triage hints* rather than definitive labels unless additional calibration or post-filtering is applied.

F. Feature Importance

The notebook reports gain-based feature rankings. For binary LightGBM, prominent features include: `init_bwd_win_bytes`, `fwd_iat_total`, `bwd_iat_min`, `init_fwd_win_bytes`, and `bwd_packets/s`.

For multi-class models, both methods strongly weight timing/header-level statistics, with repeated prominence of: `bwd_header_length`, `fwd_packet_length_std`, `packet_length_std`, `flow_iat_mean`, and `init_bwd_win_bytes`.

This aligns with an intuitive IDS interpretation: temporal burst structure, packet variability, and initial transport-window behavior capture meaningful attack signatures in flow-level telemetry.

TABLE VIII
SELECTED HIGH-GAIN FEATURES REPORTED BY THE NOTEBOOK

Binary LightGBM (examples)	Multi-class consensus (examples)
`init_bwd_win_bytes`	`bwd_header_length`
`fwd_iat_total`	`fwd_packet_length_std`
`bwd_iat_min`	`packet_length_std`
`init_fwd_win_bytes`	`flow_iat_mean`
`bwd_packets/s`	`init_bwd_win_bytes`

VII. ERROR BUDGET AND RISK FRAMING

To make model outputs actionable, we separate error types into operational buckets:

1) **Missed-attack risk** (binary false negatives): impacts containment and dwell time.
2) **Alert-load risk** (binary false positives): impacts analyst throughput.
3) **Attribution risk** (multi-class mislabeling): impacts remediation prioritization.

In this experiment, missed-attack and alert-load risks are both low at aggregate binary level, but attribution risk remains concentrated in minority classes. This supports a layered incident-response design where binary confidence gates high-priority alerting, and family labels are attached with class-specific confidence thresholds.

A. Threshold and Triage Considerations

The reported binary results are produced at a fixed threshold (0.5), but production IDS deployments rarely operate with a single static operating point. Let τ denote the alert threshold. In general:

- decreasing τ tends to increase recall and alert volume,
- increasing τ tends to reduce false alerts while increasing miss risk.

A practical policy is to maintain a high-recall primary stream and a confidence-filtered stream for automated ticket routing. The notebook's high AUC values imply that such threshold retuning can be performed with limited quality degradation because ranking quality is already strong.

B. Proposed Post-Processing for Minority Classes

The per-class report suggests that minority labels should be post-processed with explicit safeguards:

1) **Class-wise confidence floors**: require higher posterior confidence for Webattack and Infiltration labels.
2) **One-vs-rest verification**: run a lightweight secondary verifier for low-support classes.
3) **Top-k reporting**: expose the top-2 classes for analyst review when confidence gap is small.
4) **Temporal smoothing**: aggregate predictions across short flow windows to reduce bursty false positives.

These controls do not require retraining and can often be integrated at inference-service level.

VIII. Discussion

A. Operational Implications

For alerting pipelines where missed attacks are costly, both binary models provide an excellent baseline. Recall around 0.958 with AUC near 0.997 suggests reliable ranking and high sensitivity. A practical SOC deployment can still improve utility via threshold tuning and confidence calibration to control analyst load.

For attack-family attribution, aggregate performance is acceptable but uneven across classes. Weighted F1 near 0.87 can hide minority fragility, so Macro F1 and class-wise precision must be monitored in production model governance.

B. Why LightGBM Wins Macro F1 Here

Two likely factors explain LightGBM's macro advantage in this run:

- Leaf-wise growth can capture minority-specific local partitions efficiently.
- Histogram-based split behavior may better exploit sparse discriminative patterns in selected flow statistics.

However, differences are dataset- and preprocessing-dependent; future retuning could narrow or reverse this gap.

C. What the Results Suggest for Model Selection

If the primary objective is binary alert quality, either model is acceptable and can be selected by infrastructure constraints (training stack maturity, GPU access, explainability preferences). If the objective includes robust minority-class attribution, the observed Macro F1 gap favors LightGBM in this run. A practical path is to deploy one binary detector and maintain model-level A/B testing for the second-stage classifier.

IX. Reproducibility Checklist

To facilitate auditability and handoff, we summarize a minimal reproducibility checklist:

- Preserve exact train/validation/test index splits across both tasks.
- Store feature schema after column normalization and dtype downcasting.
- Persist model artifacts and feature-importance exports alongside notebook output.
- Log GPU/CPU mode explicitly per training cell.
- Record threshold values used for reporting precision/recall/F1.
- Report both macro and weighted metrics for multi-class output.
- Include class supports in every per-class table to contextualize instability.

This checklist is especially important for security research where minor preprocessing mismatches can induce large metric swings on rare classes.

X. Threats to Validity and Limitations

- **Dataset shift risk**: CIC benchmark traffic differs from live enterprise environments and may overstate transferability.
- **Temporal leakage concern**: Combined datasets from multiple years require careful split hygiene to prevent subtle leakage.
- **Flow-only features**: Payload-agnostic flow summaries improve privacy and speed, but may miss semantic attack cues.
- **Minority instability**: Extremely small classes (Portscan/Webattack) can produce volatile precision/recall behavior.
- **No latency benchmark**: This report focuses on predictive quality, not end-to-end inference throughput under streaming load.

XI. Deployment-Oriented Recommendations

1) Use a two-stage architecture: binary detector first, then conditional multi-class attribution.
2) Tune class-specific thresholds on validation data rather than fixed 0.5 cutoffs.
3) Add probability calibration and uncertainty flags for rare-family predictions.
4) Monitor drift with rolling-window recalibration and periodic retraining.
5) Explore minority-aware augmentation (reweighting, focal objectives, constrained ensembling).

XII. Future Work Directions

The notebook already demonstrates strong baseline performance. The most promising next steps are:

- **Temporal generalization**: train on earlier windows and test on later windows to stress drift.
- **Calibration**: apply post-hoc calibration (e.g., Platt or isotonic) separately for binary and multi-class outputs.
- **Hierarchical labeling**: classify Attack vs. Benign first, then subgroup attacks with tailored feature subsets.
- **Cost-aware learning**: optimize for weighted error costs that reflect SOC escalation policy.
- **Serving benchmarks**: measure throughput, latency, and memory under online inference conditions.

XIII. Extended Quantitative Results

This section presents full per-class numeric reports from the notebook outputs to preserve fidelity and support downstream audit.

A. Interpretation of Extended Tables

Three patterns emerge from Tables IX and X:

- Dominant classes (Benign, DDoS, DoS, Botnet, Bruteforce) are modeled very well by both learners.
- Tail classes exhibit high recall but unstable precision, especially Webattack.

TABLE IX
FULL LIGHTGBM MULTI-CLASS CLASSIFICATION REPORT (TEST SPLIT)

Class	Precision	Recall	F1-score	Support
Benign	1.00	0.74	0.85	1,077,929
Botnet	0.99	1.00	0.99	21,882
Bruteforce	1.00	1.00	1.00	15,347
DDoS	1.00	1.00	1.00	185,758
DoS	0.99	1.00	1.00	59,269
Infiltration	0.05	0.95	0.09	14,152
Portscan	0.69	0.95	0.80	353
Webattack	0.35	0.98	0.51	448
Accuracy	–	–	0.80	1,375,138
Macro avg	0.76	0.95	0.78	1,375,138
Weighted avg	0.99	0.80	0.87	1,375,138

TABLE X
FULL XGBOOST MULTI-CLASS CLASSIFICATION REPORT (TEST SPLIT)

Class	Precision	Recall	F1-score	Support
Benign	1.00	0.74	0.85	1,077,929
Botnet	0.98	1.00	0.99	21,882
Bruteforce	1.00	1.00	1.00	15,347
DDoS	1.00	1.00	1.00	185,758
DoS	0.99	1.00	0.99	59,269
Infiltration	0.05	0.95	0.09	14,152
Portscan	0.53	0.98	0.69	353
Webattack	0.22	0.98	0.35	448
Accuracy	–	–	0.80	1,375,138
Macro avg	0.72	0.95	0.75	1,375,138
Weighted avg	0.99	0.80	0.87	1,375,138

- Macro-level differences between learners are driven primarily by tail precision behavior, not by majority-class shifts.

This confirms that future improvements should target minority-class decision boundaries rather than global retraining without class-aware objectives.

B. From Research Metrics to SOC Workflow

A practical ingestion workflow based on these results can be structured as:

1) Score each flow with binary detector probability.
2) Route high-confidence attack flows directly to high-priority queue.
3) Run multi-class attribution only on flows above binary threshold.
4) Attach confidence and top-k alternatives to analyst-facing payload.
5) Suppress low-confidence minority labels unless corroborated by additional signals.

This converts aggregate model quality into policy controls that are easier to monitor over time.

XIV. CONCLUSION

This report demonstrates that a unified gradient boosting pipeline can deliver high-quality large-scale intrusion detection across both binary and multi-class tasks on combined CIC traffic. Binary detection is near-saturated for both LightGBM and XGBoost, while multi-class results remain constrained by rare-class precision collapse despite strong recall. LightGBM offers the best overall balance in this experiment, especially in Macro F1. Future work should focus on rare-class precision improvement, temporal robustness, and calibrated deployment in online settings.

REPRODUCIBILITY NOTES

All numeric values in tables were extracted from stored notebook outputs of the referenced workflow. The manuscript is designed as a direct transformation from executable experimentation to submission-style IEEE reporting.

REFERENCES

[1] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.

[2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[4] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.

[6] Canadian Institute for Cybersecurity, University of New Brunswick, "Intrusion detection evaluation dataset repository," https://www.unb.ca/cic/datasets/index.html, Accessed 2026.

[7] dhoogla, "Cicids collection (kaggle dataset)," https://www.kaggle.com/datasets/dhoogla/cicidscollection, Accessed 2026.